

Docker Compose with Multi-Container Applications

Start by creating a new directory for your project using the following commands: `mkdir multi-container-app` to create the directory, followed by `cd multi-container-app` to navigate into it. Inside this directory, you will organise the project by creating subdirectories or files for each component, such as the web service, Redis database, and Docker configuration, to ensure a well-structured project setup.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\MAWUKOENYA> mkdir multi-container-app

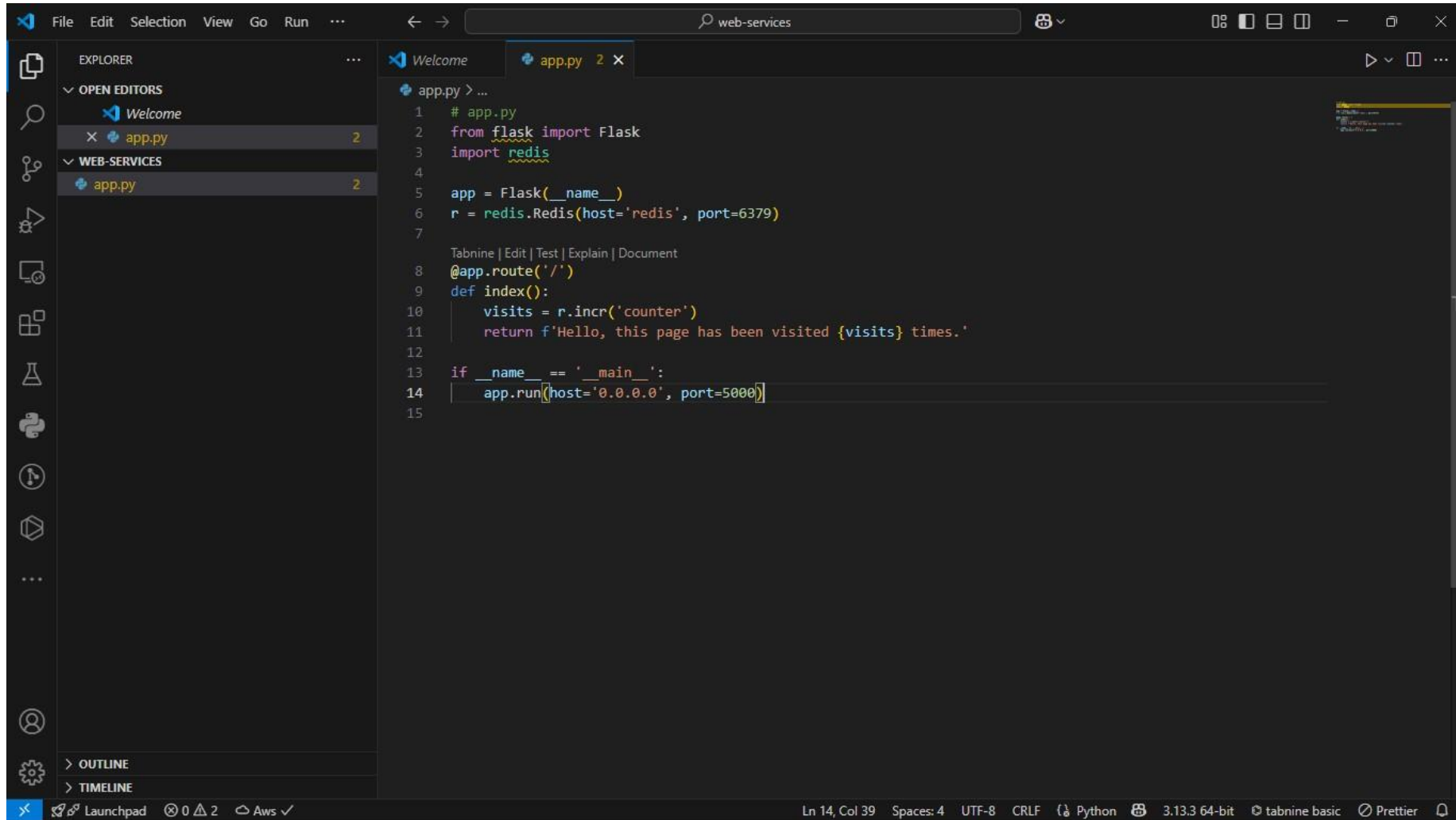
Directory: C:\Users\MAWUKOENYA

Mode                LastWriteTime         Length Name
----                -
d-----         4/27/2025   2:05 PM                multi-container-app

PS C:\Users\MAWUKOENYA> cd .\multi-container-app\
PS C:\Users\MAWUKOENYA\multi-container-app>
```

I will create a file named *app.py* that contains the Python Flask application. In this file, we will import Flask and Redis, initialise the Flask app, and set up the connection to the Redis service. The application will have a route that increments a counter stored in Redis each time

the page is accessed, displaying the number of visits. Finally, the Flask app will run on port 5000 and be accessible on all network interfaces by setting `app.run(host='0.0.0.0', port=5000)`.

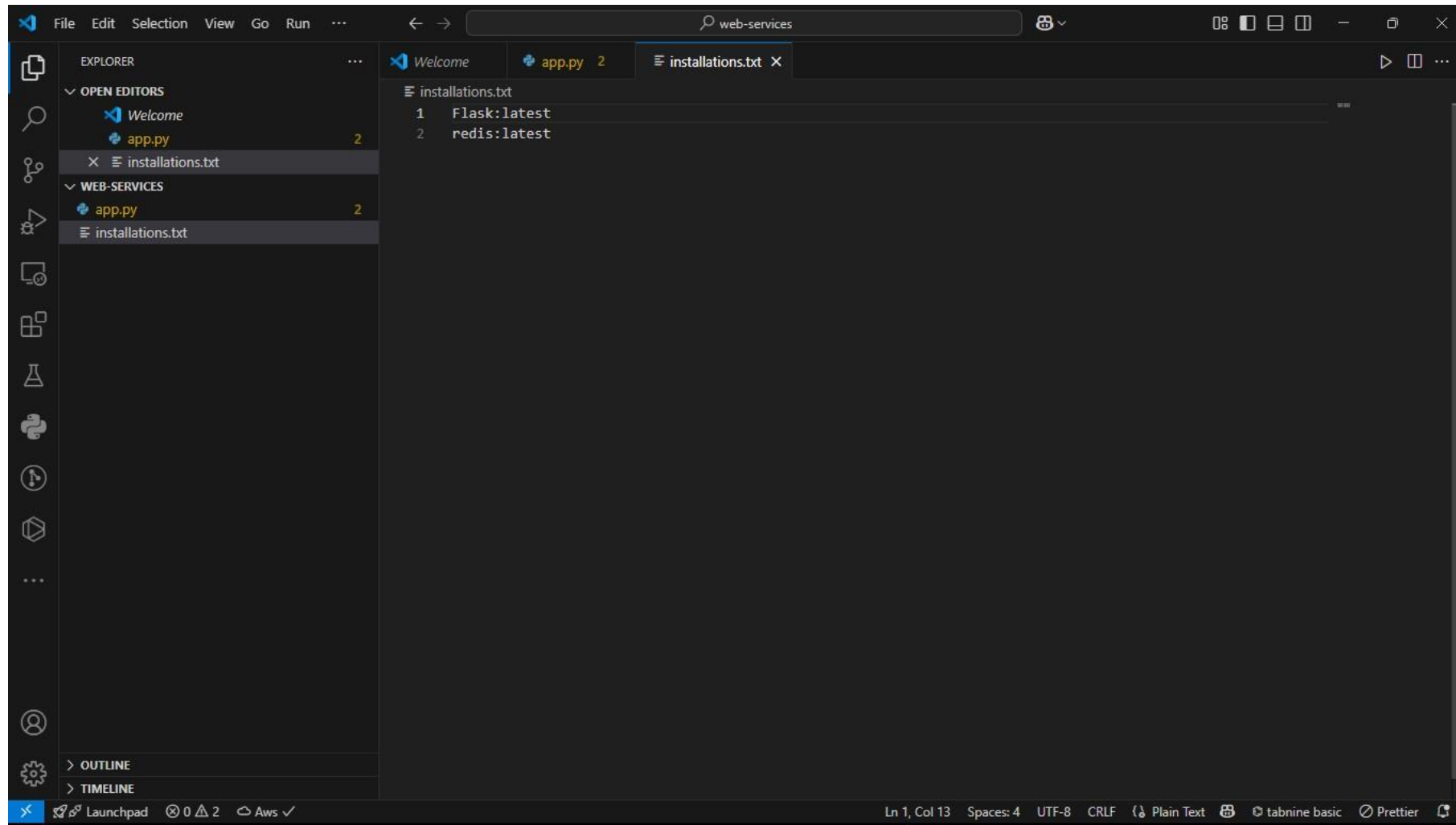


The screenshot shows the Visual Studio Code interface with a Python file named `app.py` open. The Explorer sidebar on the left shows the file structure with `app.py` under the `WEB-SERVICES` folder. The main editor area displays the following Python code:

```
1 # app.py
2 from flask import Flask
3 import redis
4
5 app = Flask(__name__)
6 r = redis.Redis(host='redis', port=6379)
7
8 @app.route('/')
9 def index():
10     visits = r.incr('counter')
11     return f'Hello, this page has been visited {visits} times.'
12
13 if __name__ == '__main__':
14     app.run(host='0.0.0.0', port=5000)
```

The status bar at the bottom indicates the file is at line 14, column 39, with 4 spaces, UTF-8 encoding, CRLF line endings, Python 3.13.3 64-bit, and the tabnine basic extension. The bottom-left corner shows the Launchpad icon, 0 errors and 2 warnings, and the AWS logo.

I will create an ***installations.txt*** file inside the web-service directory to list the dependencies for the Python Flask application. This file will include the necessary libraries that need to be installed for the web service to run correctly. In this case, the file will contain the following: Flask and Redis. These dependencies will ensure that Flask (for building the web service) and the Redis client library (for connecting to Redis) are installed when setting up the environment.

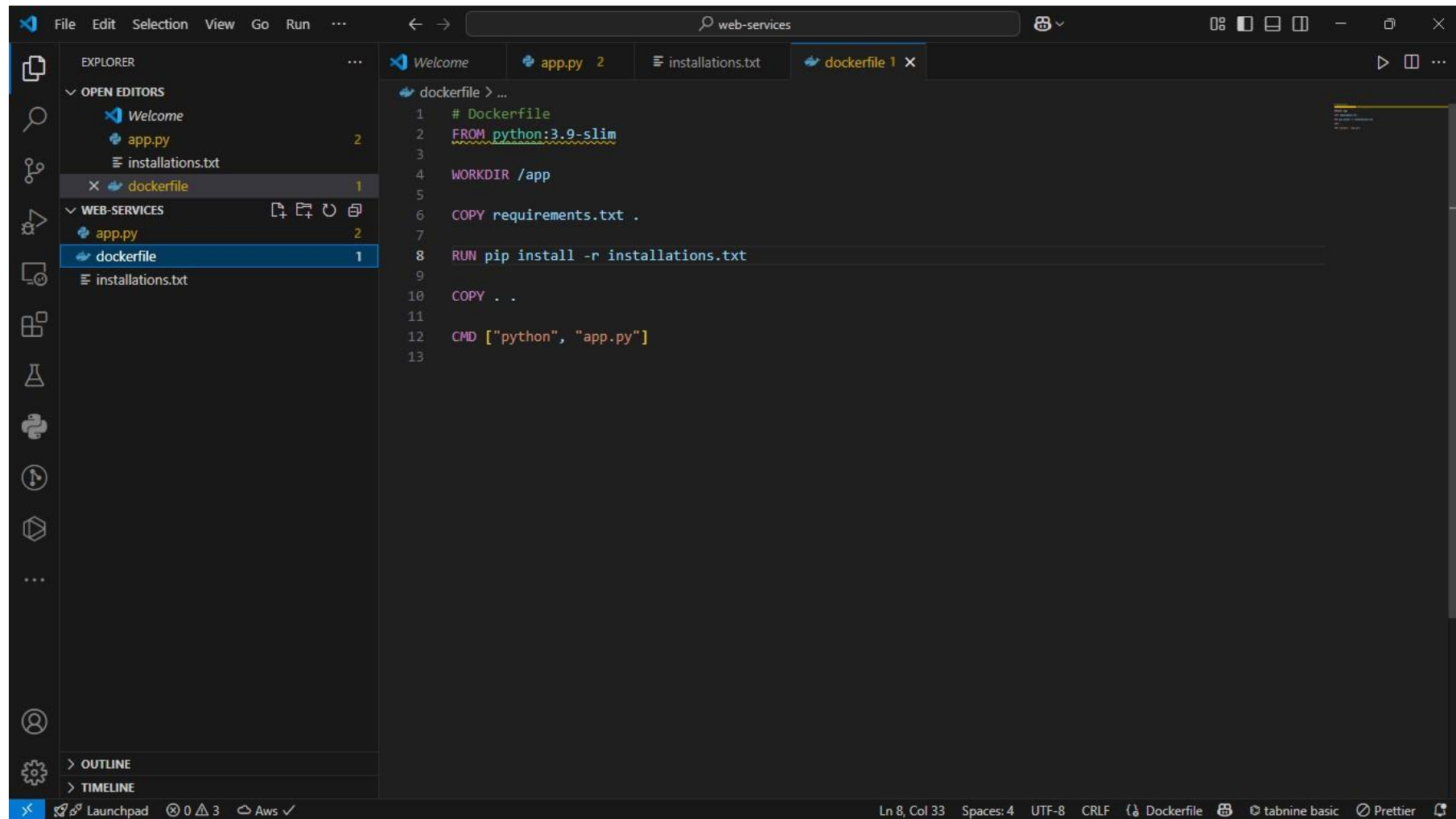


The screenshot shows the Visual Studio Code interface with a dark theme. The Explorer sidebar on the left shows the project structure with 'OPEN EDITORS' and 'WEB-SERVICES' sections. The 'installations.txt' file is open in the editor, displaying the following content:

```
installations.txt
1 Flask:latest
2 redis:latest
```

The status bar at the bottom indicates the file is at Line 1, Column 13, using UTF-8 encoding with CRLF line endings, and is formatted with Prettier.

I will create a Dockerfile to containerise the Python Flask web service. The Dockerfile will start with the official python:3.9-slim image, set the working directory to /app, and copy the *installations.txt* file into the container. It will then install the dependencies listed in requirements.txt using pip. Next, the remaining files from the local directory, including app.py, will be copied into the container. Finally, the CMD instruction will specify that the container should run the Flask application by executing python app.py when it starts.

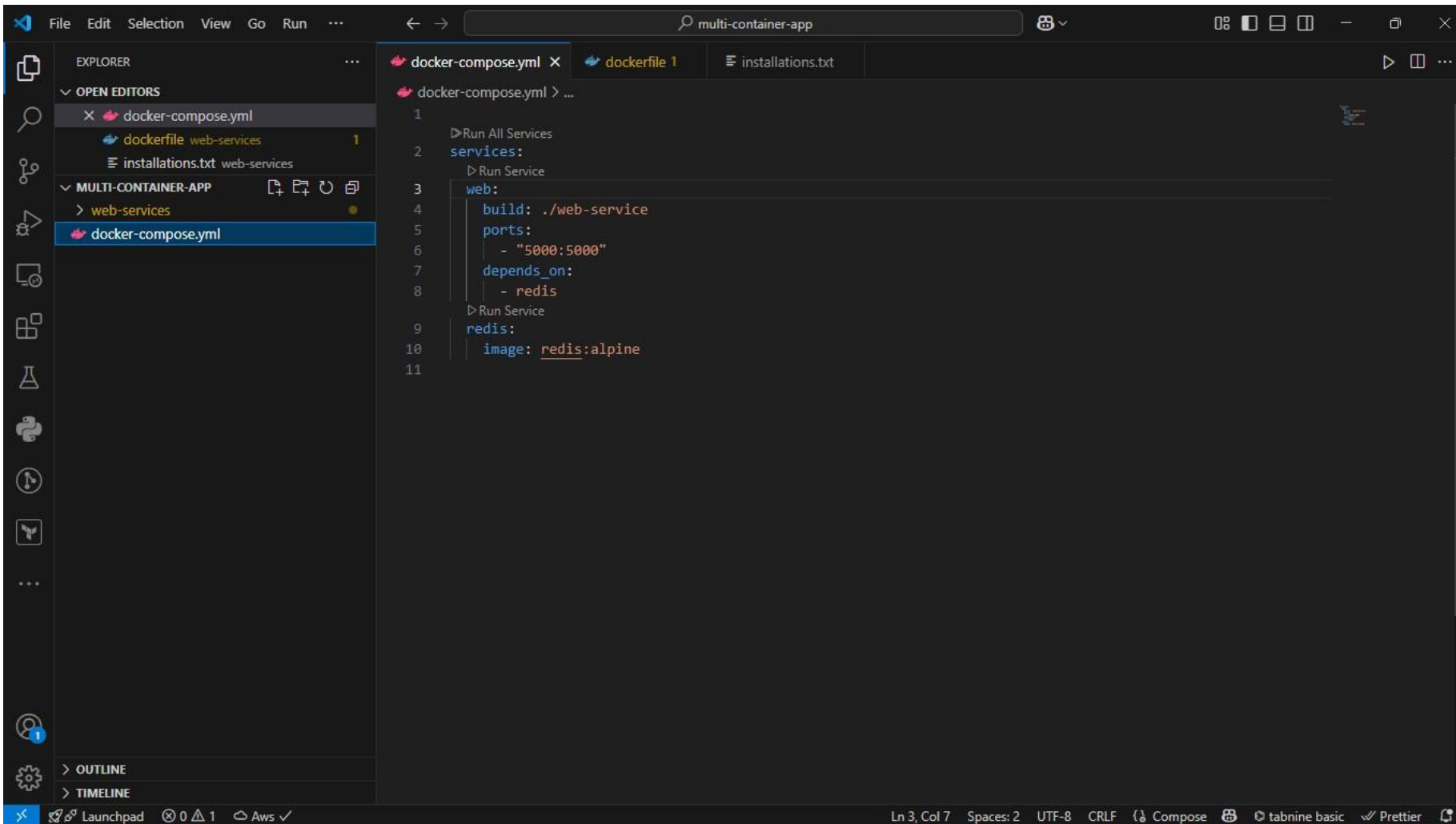


The screenshot shows the Visual Studio Code interface with a project named 'web-services'. The Explorer sidebar on the left shows the file structure: 'OPEN EDITORS' (Welcome, app.py, installations.txt, dockerfile) and 'WEB-SERVICES' (app.py, dockerfile, installations.txt). The 'dockerfile' file is selected and open in the editor. The code in the editor is as follows:

```
1 # Dockerfile
2 FROM python:3.9-slim
3
4 WORKDIR /app
5
6 COPY requirements.txt .
7
8 RUN pip install -r installations.txt
9
10 COPY . .
11
12 CMD ["python", "app.py"]
13
```

The status bar at the bottom indicates the current position is Ln 8, Col 33, with 4 spaces, UTF-8 encoding, CRLF line endings, and the Dockerfile icon. It also shows the use of the 'tabnine basic' AI assistant and the 'Prettier' formatter.

I will create a ***docker-compose.yml*** file in the project root directory to define the configuration for both the web and Redis services. In this file, the web service is built from the `./web-service` directory, exposes port 5000, and depends on the redis service. The redis service uses the official ***redis:alpine*** image. The ***depends_on*** instruction ensures that Redis starts before the web service, enabling them to communicate effectively.

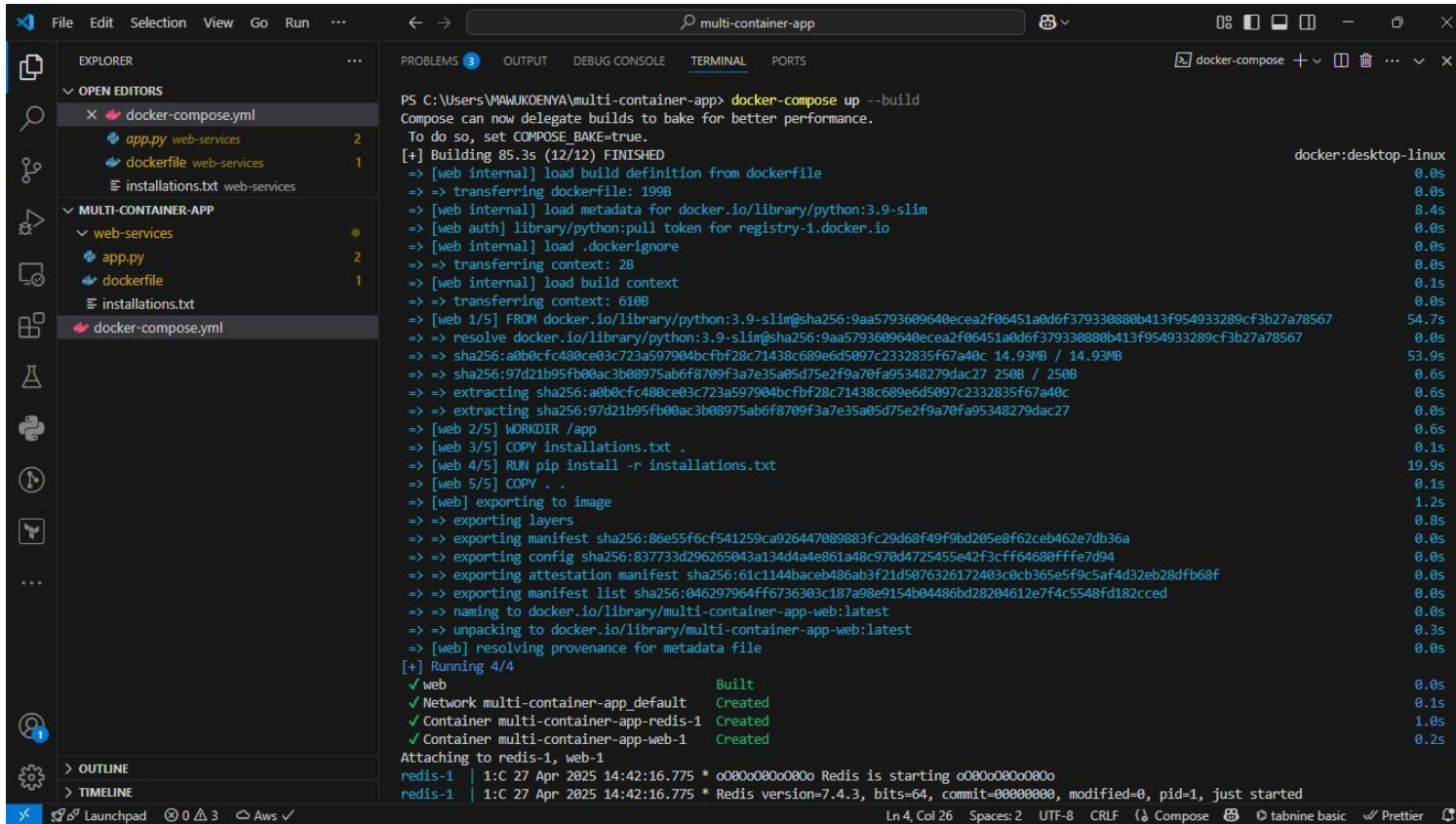


The screenshot shows the Visual Studio Code editor interface with the `docker-compose.yml` file open. The Explorer sidebar on the left shows the project structure with `docker-compose.yml` selected. The main editor area displays the following YAML configuration:

```
1  > Run All Services
2  services:
3    > Run Service
4    web:
5      build: ./web-service
6      ports:
7        - "5000:5000"
8      depends_on:
9        - redis
10   > Run Service
11   redis:
12     image: redis:alpine
```

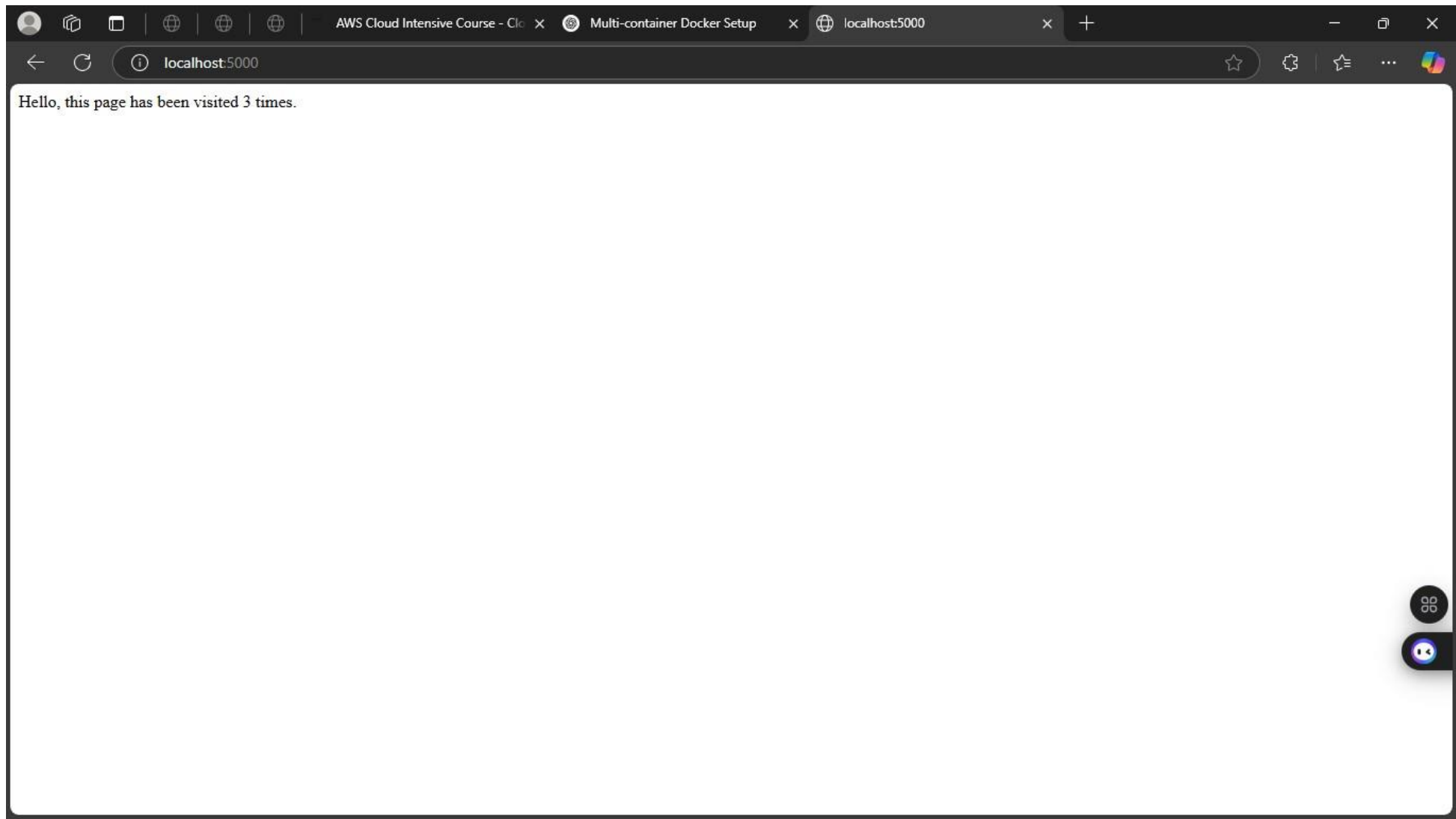
The status bar at the bottom indicates the file is at Line 3, Column 7, using UTF-8 encoding with CRLF line endings. It also shows the Compose extension is active and the Prettier formatter is applied.

I will run and test the multi-container application by navigating to the root directory (where the docker-compose.yml file is located) and running the command `docker-compose up --build`. This will build and start the containers for both the web service and Redis



```
File Edit Selection View Go Run ... multi-container-app
EXPLORER
  OPEN EDITORS
    docker-compose.yml
    app.py web-services 2
    dockerfile web-services 1
    installations.txt web-services
  MULTI-CONTAINER-APP
    web-services
    app.py 2
    dockerfile 1
    installations.txt
    docker-compose.yml
  PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
  PS C:\Users\WAMUKOENYA\multi-container-app> docker-compose up --build
  Compose can now delegate builds to bake for better performance.
  To do so, set COMPOSE_BAKE=true.
  [+] Building 85.3s (12/12) FINISHED
  => [web internal] load build definition from dockerfile
  => => transferring dockerfile: 199B
  => [web internal] load metadata for docker.io/library/python:3.9-slim
  => [web auth] library/python:pull token for registry-1.docker.io
  => [web internal] load .dockerignore
  => => transferring context: 2B
  => [web internal] load build context
  => => transferring context: 610B
  => [web 1/5] FROM docker.io/library/python:3.9-slim@sha256:9aa5793609640ecea2f06451a0d6f379330880b413f954933289cf3b27a78567
  => => resolve docker.io/library/python:3.9-slim@sha256:9aa5793609640ecea2f06451a0d6f379330880b413f954933289cf3b27a78567
  => => sha256:a0b0cfc480ce03c723a597904bcfbf28c71438c689e6d5097c2332835f67a40c 14.93MB / 14.93MB
  => => sha256:97d21b95fb00ac3b08975ab6f8709f3a7e35a05d75e2f9a70fa95348279dac27 250B / 250B
  => => extracting sha256:a0b0cfc480ce03c723a597904bcfbf28c71438c689e6d5097c2332835f67a40c
  => => extracting sha256:97d21b95fb00ac3b08975ab6f8709f3a7e35a05d75e2f9a70fa95348279dac27
  => [web 2/5] WORKDIR /app
  => [web 3/5] COPY installations.txt .
  => [web 4/5] RUN pip install -r installations.txt
  => [web 5/5] COPY . .
  => [web] exporting to image
  => => exporting layers
  => => exporting manifest sha256:86e55f6cf541259ca926447089883fc29d68f49f9bd205e8f62ceb462e7db36a
  => => exporting config sha256:837733d296265043a134d4a4e861a48c970d4725455e42f3cfff64680fffe7d94
  => => exporting attestation manifest sha256:61c1144baceb486ab3f21d5076326172403c0cb365e5f9c5af4d32eb28dfb68f
  => => exporting manifest list sha256:046297964ff6736303c187a98e9154b04486bd28204612e7f4c5548fd182cccd
  => => naming to docker.io/library/multi-container-app-web:latest
  => => unpacking to docker.io/library/multi-container-app-web:latest
  => [web] resolving provenance for metadata file
  [+] Running 4/4
  ✓ web Built 0.0s
  ✓ Network multi-container-app_default Created 0.1s
  ✓ Container multi-container-app-redis-1 Created 1.0s
  ✓ Container multi-container-app-web-1 Created 0.2s
  Attaching to redis-1, web-1
  redis-1 | 1:C 27 Apr 2025 14:42:16.775 * o000o000o000o Redis is starting o000o000o000o
  redis-1 | 1:C 27 Apr 2025 14:42:16.775 * Redis version=7.4.3, bits=64, commit=00000000, modified=0, pid=1, just started
  Launchpad 0 3 Aws ✓ Ln 4, Col 26 Spaces: 2 UTF-8 CRLF Compose tabnine basic Prettier
```

Once the containers are running, we can open a browser and visit <http://localhost:5000>, where we should see the message: "Hello, this page has been visited X times," with X representing the visit count. Each time the page is refreshed, the counter will increment, as the web service interacts with Redis to update the visit count.



CONCLUSION

In this assignment, we successfully created a multi-container application using Docker. I started by setting up the project directory and then built a simple Python Flask web service, which was containerised using Docker. I configured Redis as a database for tracking page visits and ensured that both services were linked together using Docker Compose. By running the containers with `docker-compose up-- build`, I verified that the web service and Redis could communicate and function correctly, with the visit counter being incremented each time the page was refreshed. This assignment demonstrated how to create a basic yet effective multi-container application, showcasing the power of Docker for managing service dependencies and configurations.